

micro-ROS vs. zenoh-mcu-pc-bridge

Technical Whitepaper: Topic Scaling, Custom Interfaces, Memory Overhead & Protocol Architecture

Executive Overview

Robotics developers deploying **micro-ROS** on microcontrollers (ESP32, STM32, RP2040) frequently encounter strict compile-time limits (default **4 publishers/topics**), heavy RAM consumption, complex agent dependencies, and friction when compiling custom .msg and .srv interfaces. **zenoh-mcu-pc-bridge** is a lightweight, zero-overhead alternative powered by Eclipse Zenoh-pico, MsgPack serialization, and an automated IDL generator.

Side-by-Side Comparison Matrix

Evaluation Metric	micro-ROS (Micro XRCE-DDS)	zenoh-mcu-pc-bridge (Eclipse Zenoh)
Out-of-the-Box Topic Limit	4 Publishers / 4 Topics (Statically capped via RMW_UXRCE_MAX_PUBLISHERS)	Unlimited (Constrained only by physical bus bandwidth)
Custom Interface (.msg / .srv) Creation	Tedious & Complex Requires building micro_ros_setup colcon workspace overlays and custom C IDL generators.	Effortless Auto-Generator Single command (generate.py) compiles standard ROS2 .msg/.srv files into C++ and Python headers instantly.
Practical Scaling Cap	~10 to 15 Topics Max (Causes buffer starvation and packet dropouts)	34+ Topics & Services Tested (Proven live in MCU stress testbench firmware)
Memory Allocation Strategy	Static Array Allocation (Heavy C structures per entity; static RAM bloat)	Lightweight C++ Wrappers (Shared stack buffers & zero static DDS entity bloat)
Host PC Agent Dependency	Heavy XRCE-DDS Agent (Requires multi-step XML/binary handshake)	Zero-Proxy Direct Routing (Key Expressions route directly over Wi-Fi or USB CDC)
Multi-Core Support	Single-threaded executor loops	Integrated FreeRTOS task parallelism across CPU cores

1. Custom Interface Creation: micro-ROS vs. zenoh-mcu-pc-bridge

The micro-ROS Problem: Defining custom .msg or .srv files in micro-ROS requires setting up a dedicated Linux build environment with colcon, cross-compiling the custom interfaces into a static library via micro_ros_setup, and linking C-specific IDL wrappers. This makes prototyping custom sensor schemas (IMUs, custom telemetry, motor diagnostics) time-consuming and error-prone.

The zenoh-mcu-pc-bridge Solution: Includes a built-in Python IDL generator (generate.py). Developers simply drop standard ROS2 syntax .msg or .srv files into the interface folder and run:

```
python3 generate.py -generate custom
```

This automatically generates cross-platform, MsgPack-serialized C++ headers and Python classes for MCU firmware (C++), PC C++ nodes, and PC Python nodes simultaneously.

2. Topic Scaling & Memory Architecture

Why micro-ROS Caps Out: Micro-ROS relies on static memory allocation flags (RMW_UXRCE_MAX_PUBLISHERS=4) to avoid MCU heap fragmentation. Each topic requires a full hierarchy of static C structs (rcl_publisher_t → rmw_publisher_t → uxrSession → uxrObjectId). Overriding these limits consumes tens of kilobytes of RAM for stream history buffers, leading to starvation and MCU panics.

Why zenoh-mcu-pc-bridge Scales Easily: Topics in Zenoh are lightweight string **Key Expressions** (e.g. robot/sim_counter, robot/mpu6050). Rather than creating heavy DDS session objects on the MCU, messages are serialized into reusable stack buffers and routed via Zenoh-pico over TCP Wi-Fi, High-Speed UART, or Native 12 Mbps USB CDC.

3. Empirical Verification: 34-Topic & 4-Service Stress Testbench

The testbench firmware (mcu_firmware_examples/interface_test_publisher/src/main.cpp) demonstrates this scaling in action on a single ESP32-S3 microcontroller:

Verified Testbench Interfaces (Published Live at 1 Hz):

- *27 Standard ROS2 Topics:* bool, int8-64, float32-64, string, header, multi-arrays, imu, temp, range, humidity, battery, navsatfix, joyfeedback, jointstate, vector3, quaternion, pose, twist.
- *7 Custom Messages:* set_led, mpu6050, robot_state, motor_status, telemetry, robot_diag, complex_state.
- *4 Custom Services:* set_led_color, set_color, configure_robot, full_system_control.

4. Summary & Repository Access

By replacing heavy DDS proxy handshakes with Eclipse Zenoh-pico, MsgPack framing, and FreeRTOS task concurrency, **zenoh-mcu-pc-bridge** provides a clean, high-performance, developer-friendly communication bridge for next-generation MCU robotics.

GitHub Repository: <https://github.com/CoreBotics-Lab/zenoh-mcu-pc-bridge>